

Atividades de Expressões Atribuição e Controle

Aula 07 Paradigmas de Linguagens de Programação

Matheus Previato Hermenegildo

R.A.: 24117297-2

ESOFT6SB

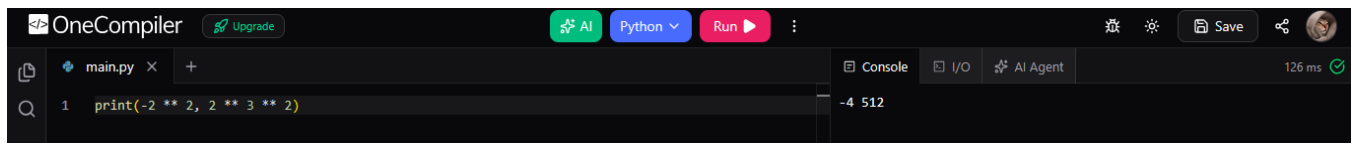
Quiz de abertura

Para cada trecho, o palpite foi registrado antes da execução. Depois, a saída foi conferida no OneCompiler e explicada pela regra da linguagem.

1- Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

Palpite antes de executar: Número Positivo

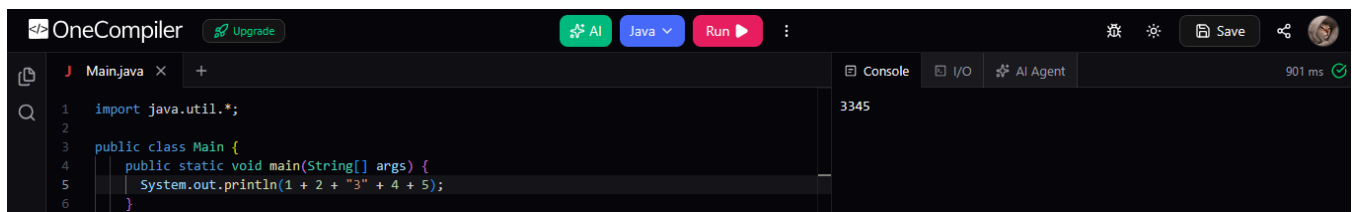


The screenshot shows the OneCompiler interface with the Python tab selected. The code `print(-2 ** 2, 2 ** 3 ** 2)` is entered in the editor. The console output shows `-4 512` with a green checkmark indicating successful execution. The execution time is 126 ms.

2 - Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

Palpite antes de executar: 3345

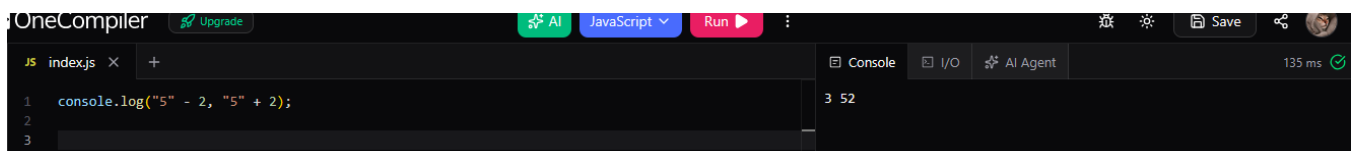


The screenshot shows the OneCompiler interface with the Java tab selected. The code `System.out.println(1 + 2 + "3" + 4 + 5);` is entered in the editor. The console output shows `3345` with a green checkmark indicating successful execution. The execution time is 901 ms.

3 - JavaScript

```
console.log("5" - 2, "5" + 2);
```

Palpite antes de executar: 3 7



The screenshot shows the OneCompiler interface with the JavaScript tab selected. The code `console.log("5" - 2, "5" + 2);` is entered in the editor. The console output shows `3 52` with a green checkmark indicating successful execution. The execution time is 135 ms.

4 - C

```
printf("%d\n", 3 > 2 > 1);
```

Palpite antes de executar: 0



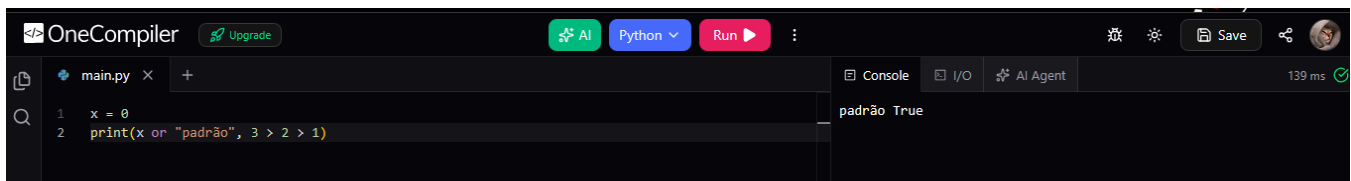
```
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
```

Console: 0

5 - Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

Palpite antes de executar: padrão True



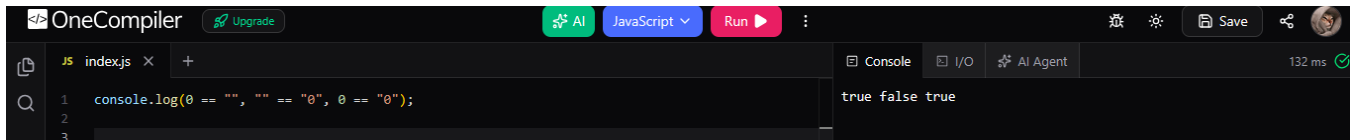
```
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
```

Console: padrão True

6 - JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

Palpite antes de executar: true false true



```
1 console.log(0 == "", "" == "0", 0 == "0");
2
3
```

Console: true false true

Expressões aritméticas e questões de projeto

O projeto de uma linguagem precisa definir como uma expressão será interpretada antes que o programa possa executá-la. As principais decisões são as seguintes.

Regras de precedência

Definem quais operadores são aplicados primeiro quando aparecem níveis diferentes na mesma expressão. Muitas linguagens seguem a matemática, com exponenciação antes de multiplicação e divisão, que vêm antes de adição e subtração. APL é uma exceção: usa um único nível e avalia da direita para a esquerda.

Regras de associatividade

Definem o agrupamento de operadores com a mesma precedência. Os operadores +, -, * e / normalmente associam à esquerda. A exponenciação em Python e JavaScript associa à direita, portanto $2 ** 3 ** 2$ equivale a $2 ** (3 ** 2)$.

Ordem de avaliação dos operandos

Indica em que ordem os valores e as chamadas de função são obtidos. Java garante a avaliação da esquerda para a direita. Em C, parte dessa ordem não é definida pelo padrão, o que permite otimizações, mas torna perigoso misturar leituras e efeitos colaterais na mesma expressão.

Restrições a efeitos colaterais

Um efeito colateral ocorre quando a avaliação altera uma variável, um parâmetro ou um estado externo. Linguagens imperativas normalmente permitem esses efeitos. Linguagens funcionais puras, como Haskell, restringem esse comportamento e tornam a avaliação mais previsível. Common Lisp permite efeitos colaterais, apesar de oferecer um estilo funcional.

Sobrecarga de operadores

Permite usar ou redefinir operadores para tipos diferentes. C++, C#, Python, Rust e Kotlin aceitam sobrecarga definida pelo programador. Java não permite essa redefinição, embora use + internamente tanto para soma quanto para concatenação de Strings.

Mistura de tipos

Linguagens como C e Java realizam algumas coerções implícitas em expressões de modo misto. Go, Rust, ML e F# exigem conversões explícitas em muitos casos. A conversão explícita tende a deixar a intenção mais clara e reduz erros escondidos por coerções automáticas.

Estruturas de controle necessárias

Böhm e Jacopini demonstraram que três estruturas são suficientes para representar qualquer algoritmo: sequência, seleção e iteração. A sequência executa sentenças em ordem. A seleção escolhe um caminho de acordo com uma condição. A iteração repete sentenças enquanto uma condição for verdadeira. Estruturas como for, switch, do-while e foreach não aumentam o poder de computação, mas podem melhorar a facilidade de escrita e a legibilidade.

Laços controlados por contador

As principais questões de projeto para laços controlados por contador são:

- Tipo e escopo da variável de controle: a linguagem define quais tipos podem controlar o laço e se a variável continua acessível depois dele.
- Alteração no corpo: em C, mudar a variável dentro do corpo altera a contagem. Em Python, alterar i não muda o próximo valor entregue pelo range.
- Avaliação dos parâmetros: em C, a condição e a atualização são reavaliadas a cada volta. Em estruturas baseadas em iteráveis, a linguagem obtém os valores do objeto percorrido conforme as regras do iterador.

Exercício encontre o bug 1 de 3

Questão 1 JavaScript

Código original

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

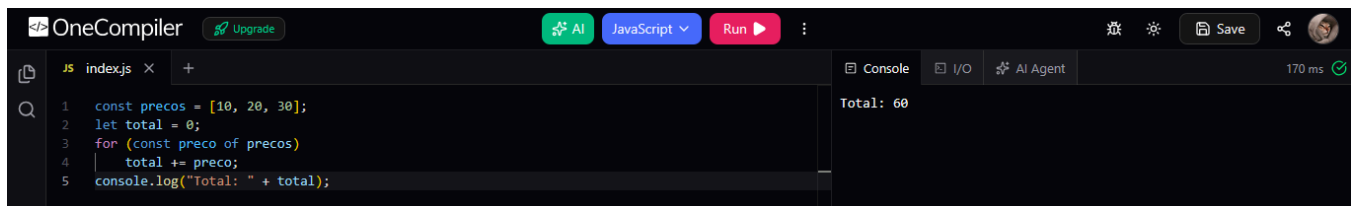
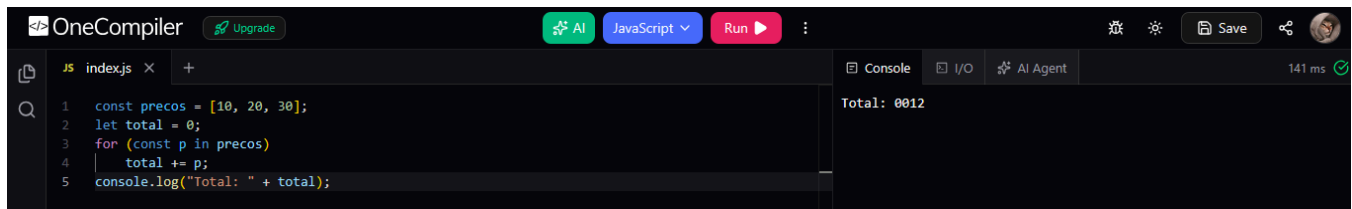
a **Saída:** Total: 0012

b **Regra que causa o problema:** for...in percorre as chaves do array. As chaves são Strings, então o operador + concatena 0, 0, 1 e 2.

c **Como corrigir:** Trocar for...in por for...of para percorrer os valores 10, 20 e 30.

Código corrigido

```
const precos = [10, 20, 30];
let total = 0;
for (const preco of precos)
  total += preco;
console.log("Total: " + total);
```



Questão 2 Python

Código original

```
def desconto(preco, d=None):
    d = d or 10
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

a **Saída:** 180.0

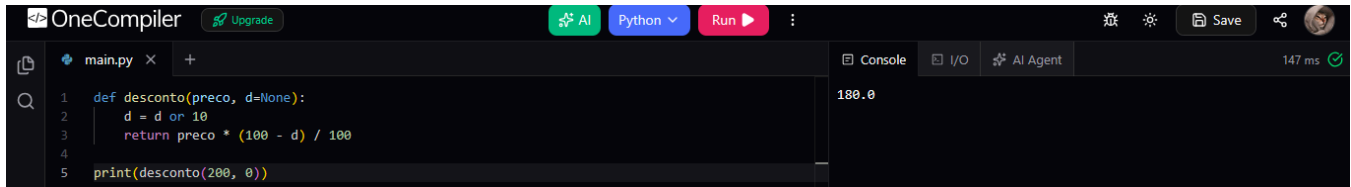
b **Regra que causa o problema:** O valor 0 é considerado falso. A expressão d or 10 substitui o desconto zero pelo padrão de 10.

c **Como corrigir:** Testar especificamente se d is None antes de aplicar o valor padrão.

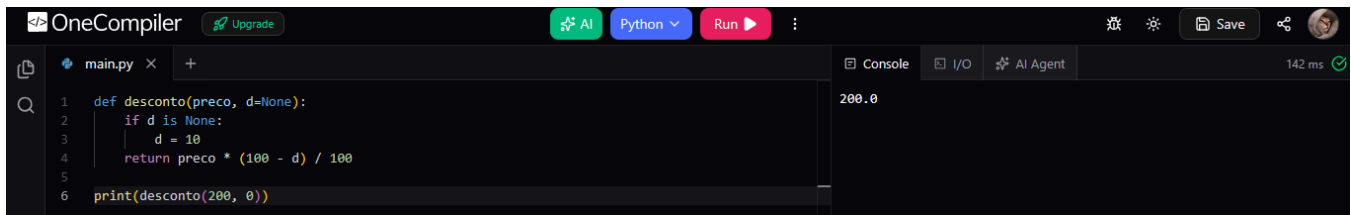
Código corrigido

```
def desconto(preco, d=None):
    if d is None:
        d = 10
    return preco * (100 - d) / 100

print(desconto(200, 0))
```



The screenshot shows the OneCompiler IDE interface. The code editor on the left contains the corrected Python code. The console on the right shows the output '180.0'. The top bar includes buttons for AI, Python, Run, and Save, along with a status bar indicating 147 ms execution time.



The screenshot shows the OneCompiler IDE interface with the original Python code. The code editor on the left contains the original code. The console on the right shows the output '200.0'. The top bar includes buttons for AI, Python, Run, and Save, along with a status bar indicating 142 ms execution time.

Exercício encontre o bug 2 de 3

Questão 3 Java

Código original

```
public class Main {
    public static void main(String[] args) {
        int dia = 2;
        String nome = "";
        switch (dia) {
            case 1: nome = "domingo";
            case 2: nome = "segunda";
            case 3: nome = "terça";
            default: nome = "inválido";
        }
        System.out.println(nome);
    }
}
```

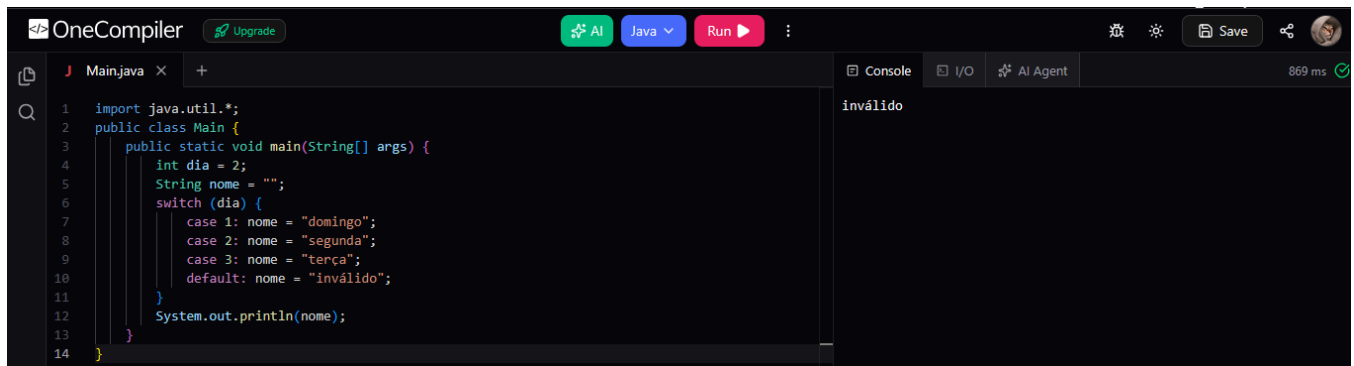
a Saída: inválido

b Regra que causa o problema: O switch tradicional continua nos casos seguintes quando não encontra break. O case 2 cai no case 3 e depois no default.

c Como corrigir: Adicionar break em cada case ou usar a sintaxe de switch com setas em versões recentes do Java.

Código corrigido

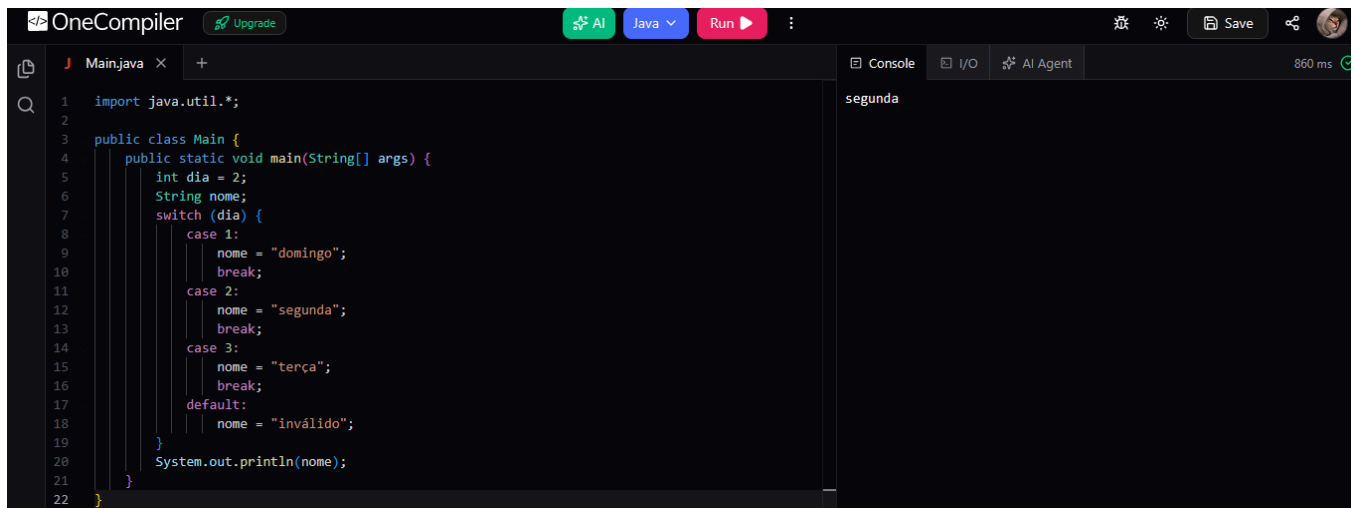
```
public class Main {  
    public static void main(String[] args) {  
        int dia = 2;  
        String nome;  
        switch (dia) {  
            case 1:  
                nome = "domingo";  
                break;  
            case 2:  
                nome = "segunda";  
                break;  
            case 3:  
                nome = "terça";  
                break;  
            default:  
                nome = "inválido";  
        }  
        System.out.println(nome);  
    }  
}
```



The screenshot shows the OneCompiler IDE interface. The code editor on the left contains the corrected Java code. The console on the right displays the output 'inválido'. The top bar includes buttons for AI, Java, Run, and Save. The bottom bar shows the file name 'Main.java' and a search icon.

```
1 import java.util.*;  
2 public class Main {  
3     public static void main(String[] args) {  
4         int dia = 2;  
5         String nome = "";  
6         switch (dia) {  
7             case 1: nome = "domingo";  
8             case 2: nome = "segunda";  
9             case 3: nome = "terça";  
10            default: nome = "inválido";  
11        }  
12        System.out.println(nome);  
13    }  
14 }
```

Console: inválido 869 ms



The screenshot shows the OneCompiler IDE interface. The code editor on the left contains the corrected Java code. The console on the right displays the output 'segunda'. The top bar includes buttons for AI, Java, Run, and Save. The bottom bar shows the file name 'Main.java' and a search icon.

```
1 import java.util.*;  
2 public class Main {  
3     public static void main(String[] args) {  
4         int dia = 2;  
5         String nome;  
6         switch (dia) {  
7             case 1:  
8                 nome = "domingo";  
9                 break;  
10            case 2:  
11                nome = "segunda";  
12                break;  
13            case 3:  
14                nome = "terça";  
15                break;  
16            default:  
17                nome = "inválido";  
18        }  
19        System.out.println(nome);  
20    }  
21 }  
22 }
```

Console: segunda 860 ms

Questão 4 C

Código original

```
#include <stdio.h>

int main(void) {
    int saldo = 100, saque = 50;
    if (saque > saldo);
        printf("saldo insuficiente\n");
    printf("fim\n");
    return 0;
}
```

a Saída: saldo insuficiente, seguido de fim, cada mensagem em uma linha

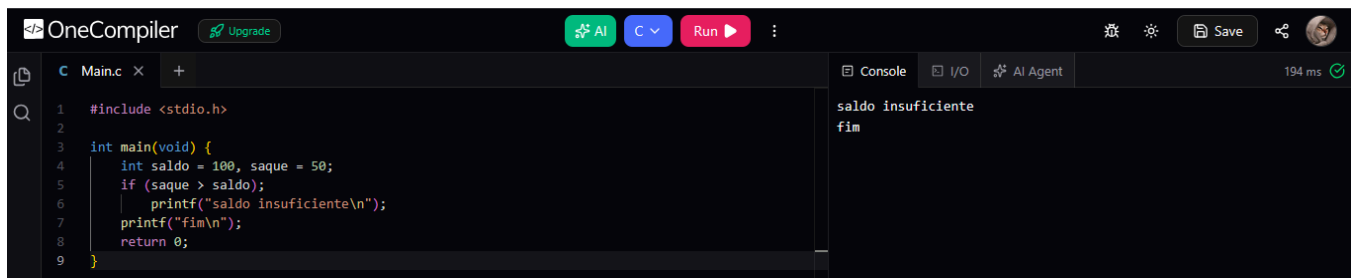
b Regra que causa o problema: O ponto e vírgula depois do if forma uma sentença vazia. O printf fica fora da condição e sempre executa.

c Como corrigir: Remover o ponto e vírgula e usar chaves para delimitar claramente o bloco do if.

Código corrigido

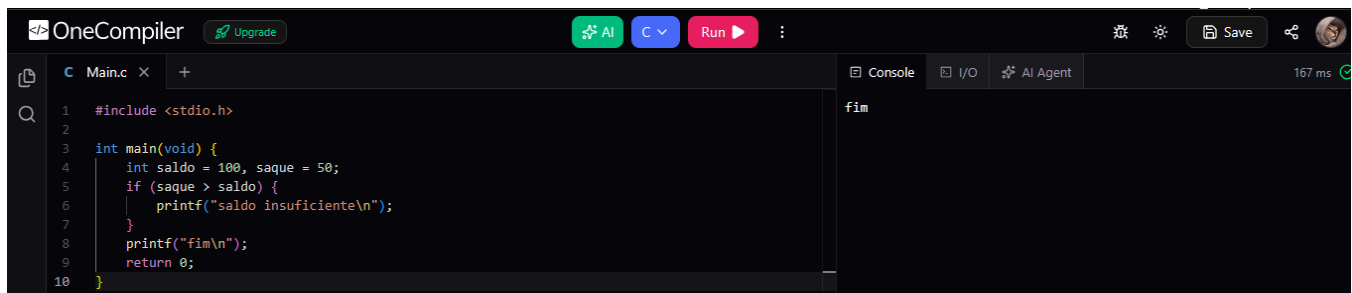
```
#include <stdio.h>

int main(void) {
    int saldo = 100, saque = 50;
    if (saque > saldo) {
        printf("saldo insuficiente\n");
    }
    printf("fim\n");
    return 0;
}
```



```
1 #include <stdio.h>
2
3 int main(void) {
4     int saldo = 100, saque = 50;
5     if (saque > saldo);
6     printf("saldo insuficiente\n");
7     printf("fim\n");
8     return 0;
9 }
```

Console: saldo insuficiente
fim



```
1 #include <stdio.h>
2
3 int main(void) {
4     int saldo = 100, saque = 50;
5     if (saque > saldo) {
6         printf("saldo insuficiente\n");
7     }
8     printf("fim\n");
9     return 0;
10 }
```

Console: fim

Exercício encontre o bug 3 de 3

Questão 5 Go

Código original

```
package main

import "fmt"

func main() {
    x := 10
    if x > 5 {
        x := x * 2
        fmt.Println("dentro:", x)
    }
    fmt.Println("fora:", x)
}
```

a Saída: dentro: 20 e fora: 10

b Regra que causa o problema: O operador `:=` dentro do `if` declara uma nova variável `x` no escopo interno. Essa variável sombrea a variável externa.

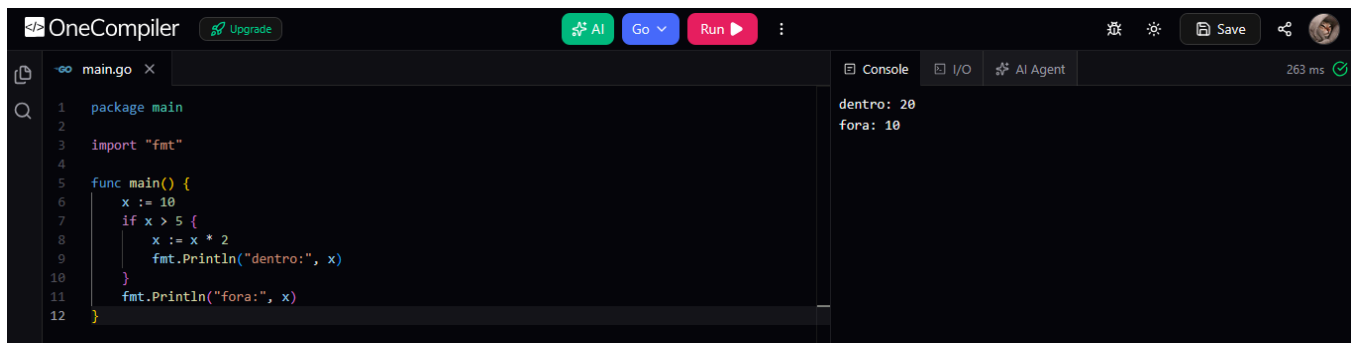
c Como corrigir: Usar `x = x * 2` para atribuir o novo valor à variável já existente.

Código corrigido

```
package main

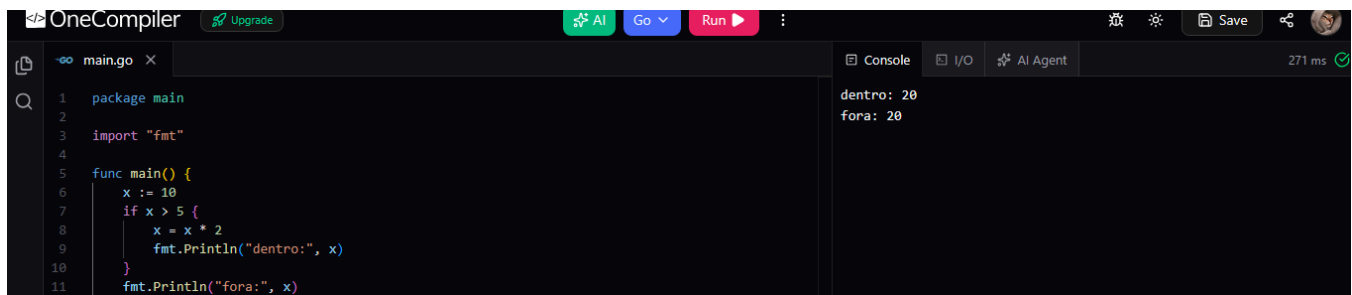
import "fmt"

func main() {
    x := 10
    if x > 5 {
        x = x * 2
        fmt.Println("dentro:", x)
    }
    fmt.Println("fora:", x)
}
```



```
1 package main
2
3 import "fmt"
4
5 func main() {
6     x := 10
7     if x > 5 {
8         x := x * 2
9         fmt.Println("dentro:", x)
10    }
11    fmt.Println("fora:", x)
12 }
```

dentro: 20
fora: 10



```
1 package main
2
3 import "fmt"
4
5 func main() {
6     x := 10
7     if x > 5 {
8         x = x * 2
9         fmt.Println("dentro:", x)
10    }
11    fmt.Println("fora:", x)
12 }
```

dentro: 20
fora: 20

Questão 6 Java

Código original

```
public class Main {
    public static void main(String[] args) {
        int acertos = 7, total = 10;
        double taxa = acertos / total * 100;
        System.out.println(taxa + "%");
    }
}
```

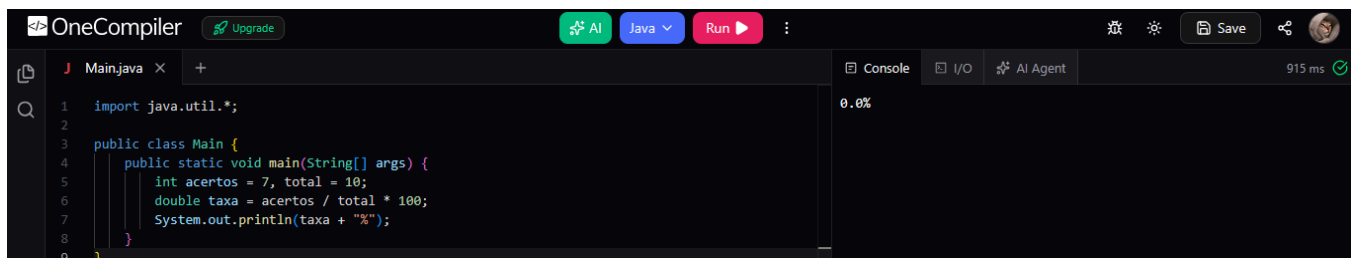
a Saída: 0.0%

b Regra que causa o problema: acertos e total são inteiros. A divisão $7 / 10$ produz 0 antes que o resultado seja convertido para double.

c Como corrigir: Introduzir um operando real antes da divisão, por exemplo $\text{acertos} * 100.0 / \text{total}$.

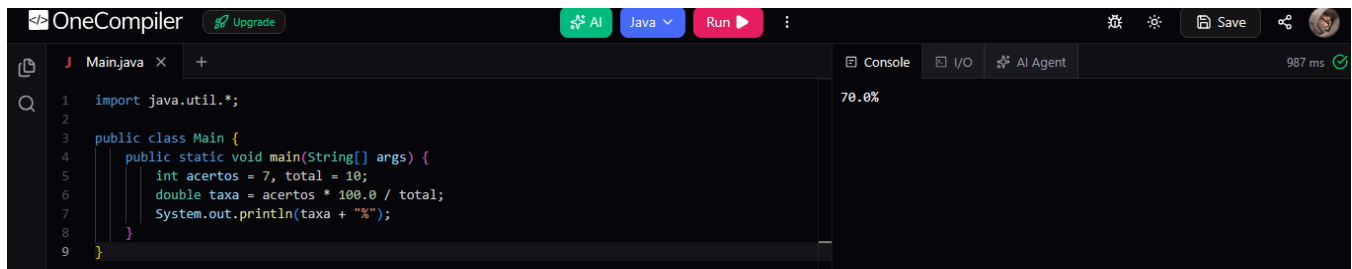
Código corrigido

```
public class Main {
    public static void main(String[] args) {
        int acertos = 7, total = 10;
        double taxa = acertos * 100.0 / total;
        System.out.println(taxa + "%");
    }
}
```



```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int acertos = 7, total = 10;
6         double taxa = acertos / total * 100;
7         System.out.println(taxa + "%");
8     }
9 }
```

0.0%



The screenshot shows the OneCompiler web IDE interface. The top bar includes the 'OneCompiler' logo, an 'Upgrade' button, and tabs for 'AI', 'Java', and 'Run'. The main editor displays a Java file named 'Main.java' with the following code:

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int acertos = 7, total = 10;
6         double taxa = acertos * 100.0 / total;
7         System.out.println(taxa + "%");
8     }
9 }
```

On the right side, the 'Console' tab is active, showing the execution progress at '70.0%' with a '987 ms' execution time and a green checkmark icon.

Conclusão

Os exemplos mostram que pequenos detalhes de precedência, coerção, escopo e controle de fluxo mudam completamente o resultado de um programa. A forma mais segura de evitar esses erros é deixar a intenção explícita com parênteses, conversões controladas, chaves e estruturas adequadas para cada linguagem.